

Databases and SQL

SQL 3 -- CRUD 2

Presented By: Rehber Moin

Agenda

- Syntax Deep-Dive (continued):
 - DISTINCT
 - ROUND()
 - WHERE
 - AND, OR, and NOT
 - IN
 - ORDER BY (including with DISTINCT)
 - LIKE (Pattern Matching)
 - NULL in SQL
 - LIMIT and OFFSET
 - UPDATE
 - DELETE, DROP, and TRUNCATE

*This class uses the **Sakila** database. Make sure it is installed.*

```
USE sakila;
```

DISTINCT -- Unique Entries

DISTINCT removes duplicate values from the result set. Internally, it works like a **Set** -- only unique entries are kept.

Syntax

```
SELECT DISTINCT column1, column2 FROM table_name;
```

This returns only unique **combinations** of the listed columns.

Example: All Different Rental Rates

```
SELECT DISTINCT rental_rate FROM film;
```

Result:

```
+-----+
| rental_rate |
+-----+
|         0.99 |
|         2.99 |
```

```
|      4.99 |
+-----+
```

DISTINCT on Multiple Columns

```
SELECT DISTINCT rental_rate, length FROM film;
```

This returns every unique **(rental_rate, length)** pair -- not unique rental rates and unique lengths separately.

Why This Does NOT Work

```
-- INVALID:
SELECT rental_rate, DISTINCT length FROM film;
```

Reason: `DISTINCT` applies to the **entire row** of selected columns, not to a single column mid-query. If `rental_rate` has entries equal to the full table size, and `DISTINCT length` has fewer entries, the column counts wouldn't match -- it's not logically possible.

*`DISTINCT` must come right after `SELECT`, and it applies to **all** columns listed.*

ROUND() -- Rounding Numbers

The `ROUND()` function rounds a numeric value to a specified number of decimal places.

Syntax

```
ROUND(number, decimal_places)
```

Example

```
SELECT ROUND(1234.567, 2) AS rounded_num;
```

Result:

```
+-----+
| rounded_num |
+-----+
|    1234.57 |
+-----+
```

WHERE -- Filtering Rows

The `WHERE` clause filters rows based on a condition. Think of it like an `if` statement inside a loop:

```
// Java equivalent of WHERE
for (auto row : table) {
    if (condition) {    // <-- This is the WHERE clause
        print(row);
    }
}
```

Syntax

```
SELECT columns FROM table WHERE condition;
```

Example: Films with Rating PG-13

```
SELECT * FROM film WHERE rating = 'PG-13';
```

Not Equal

```
-- Find all films NOT rated PG-13
SELECT * FROM film WHERE rating != 'PG-13';

-- You can also use <>
SELECT * FROM film WHERE rating <> 'PG-13';
```

AND, OR & NOT -- Logical Operators

Operator Precedence

AND has higher precedence than **OR** (just like `*` is evaluated before `+` in math).

The Trap

```
SELECT * FROM film
WHERE rating = 'PG-13' OR rental_rate = 0.99 AND release_year = 2006;
```

This is interpreted as:

```
WHERE rating = 'PG-13' OR (rental_rate = 0.99 AND release_year = 2006)
```

NOT as:

```
WHERE (rating = 'PG-13' OR rental_rate = 0.99) AND release_year = 2006
```

Rule: Always use parentheses when combining multiple logical operators to make your intent explicit.

Example with Parentheses

```
-- Films rated PG-13 OR (rental_rate = 0.99 AND released in 2006)
SELECT * FROM film
WHERE rating = 'PG-13' OR (rental_rate = 0.99 AND release_year = 2006);

-- (Films rated PG-13 OR rental_rate = 0.99) AND released in 2006
SELECT * FROM film
WHERE (rating = 'PG-13' OR rental_rate = 0.99) AND release_year = 2006;
```

These return **different** results. The parentheses matter.

IN -- Syntactic Sugar for Multiple ORs

`IN` is shorthand for multiple `OR` conditions on the same column. It works **only for equality comparisons**.

Without IN (Verbose)

```
SELECT title, rental_duration FROM film
WHERE rental_duration = 3
      OR rental_duration = 4
      OR rental_duration = 6;
```

With IN (Clean)

```
SELECT title, rental_duration FROM film
WHERE rental_duration IN (3, 4, 6);
```

Both queries return the same result.

NOT IN

```
-- All durations that are NOT 3, 4, or 6
SELECT title, rental_duration FROM film
WHERE rental_duration NOT IN (3, 4, 6);
```

ORDER BY -- Sorting Results

By default, MySQL returns rows in **ascending order of the Primary Key** (because the clustered index is built on the PK).

Syntax

```
SELECT columns FROM table ORDER BY column [ASC|DESC];
```

- `ASC` = Ascending (default, can be omitted)
- `DESC` = Descending

Examples

```
-- Default order (by Primary Key, ascending)
SELECT * FROM film;

-- Ordered by title, descending (reverse alphabetical)
SELECT * FROM film ORDER BY title DESC;
```

Sorting by Multiple Columns

```
-- Sort by rental_duration ASC first, then by title ASC (tie-breaker)
SELECT rental_duration, title FROM film
ORDER BY rental_duration, title;
```

How it works:

rental_duration		title
3		ALIEN CENTER
3		ARACHNOPHOBIA ROLLERCOASTER
3		BEDAZZLED MARRIED
...		
4		ACE GOLDFINGER
4		ADAPTATION HOLES
...		

Rows are sorted by `rental_duration` first. Within the same `rental_duration`, they are sorted by `title`.

Mixed Sort Directions

```
-- rental_duration ASC, title DESC
SELECT rental_duration, title FROM film
ORDER BY rental_duration, title DESC;

-- rental_duration DESC, title ASC
SELECT rental_duration, title FROM film
ORDER BY rental_duration DESC, title;
```

Each column gets its **own** sort direction.

Tip: If there is no tie-breaker for the `ORDER BY`, MySQL uses the Primary Key as the final tie-breaker.

Assignment: Films Rated PG-13, Sorted by Title

```
SELECT title, rating FROM film
WHERE rating = 'PG-13'
ORDER BY title;
```

Filter Then Sort vs Sort Then Filter?

If we were to do this in Java, there are two approaches:

1. Sort the entire dataset, then filter.
2. Filter first, then sort.

Which is better? Filter first, then sort. Sorting on a **reduced** number of rows is faster. SQL query optimizers do this under the hood as well.

ORDER BY with DISTINCT

You can use `ORDER BY` with `DISTINCT`, but the `ORDER BY` column **must** be one of the columns in the `SELECT DISTINCT` list.

This Works

```
SELECT DISTINCT title FROM film
ORDER BY title DESC;
```

Why Can't You ORDER BY a Non-Selected Column with DISTINCT?

Consider:

```
-- INVALID in strict mode:
SELECT DISTINCT title FROM film
ORDER BY rental_rate;
```

Reason: After `DISTINCT` removes duplicates, each row only has `title`. MySQL doesn't know which `rental_rate` to use for ordering because the `rental_rate` column was collapsed away during deduplication. There could be multiple different `rental_rate` values for the same `title`, and MySQL can't decide which one to sort by.

NULL in SQL

NULL is NOT the Same as null in Java/Python

In most programming languages, `null` means "no reference" or "nothing". In SQL, `NULL` means **unknown** or **missing value**. It behaves differently:

The Surprise

```
SELECT 3 = 3;      -- Returns: 1 (TRUE)
SELECT 3 != 3;     -- Returns: 0 (FALSE)
```

```
SELECT 3 = NULL;      -- Returns: NULL (not TRUE, not FALSE!)
```

Any comparison with **NULL** returns **NULL** (which is treated as "not true").

Why This Doesn't Work

```
SELECT * FROM film WHERE title != NULL;
-- Returns: EMPTY result set!
```

Reason: `WHERE` only returns rows where the condition is **TRUE**. When comparing against `NULL`, the result is `NULL` (unknown), which is not TRUE. So **no rows** pass the filter.

The Fix: IS NULL / IS NOT NULL

```
-- Find rows where title IS NOT NULL
SELECT * FROM film WHERE title IS NOT NULL;

-- Find rows where a column IS NULL
SELECT * FROM address WHERE address2 IS NULL;
```

Rule: Always use `IS NULL` or `IS NOT NULL` for NULL comparisons. Never use `=` or `!=` with `NULL`.

LIKE -- Pattern Matching

`LIKE` allows you to search for patterns in string columns. Think of it as a simple state machine / pattern matcher.

Wildcards

Wildcard	Meaning	Example
%	Matches zero or more characters	'D%' = starts with D
_	Matches exactly one character	'___e' = 4 chars ending in e

Examples

```
-- Films starting with the letter 'D'
SELECT * FROM film WHERE title LIKE 'D%';

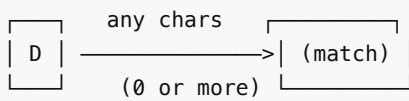
-- Films with 'Love' anywhere in the title
SELECT * FROM film WHERE title LIKE '%Love%';

-- Films ending with 'e'
SELECT * FROM film WHERE title LIKE '%e';

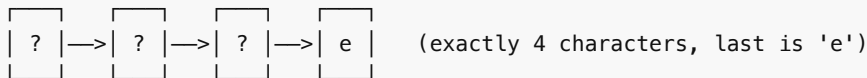
-- Films with a title exactly 4 characters long, ending with 'e'
SELECT * FROM film WHERE title LIKE '___e';
```

Pattern Diagram

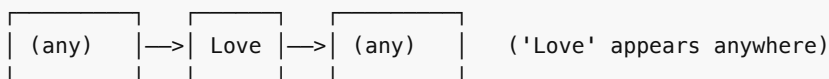
Pattern: 'D%'



Pattern: '___e'



Pattern: '%Love%'



Handling NULL with Filters

When a column is **nullable**, always account for NULL in your WHERE conditions.

Example

```
-- Find all students with PSP not equal to 80
-- WRONG (misses rows where PSP is NULL):
SELECT * FROM students WHERE psp != 80;

-- CORRECT (includes rows where PSP is NULL):
SELECT * FROM students WHERE psp != 80 OR psp IS NULL;
```

Rule: If a column is nullable, always handle **NULL** explicitly in your queries.

LIMIT and OFFSET -- Pagination

LIMIT

Restricts the number of rows returned.

```
-- Return only the first 10 films
SELECT * FROM film LIMIT 10;
```

LIMIT with ORDER BY

LIMIT comes **after** **ORDER BY** :

```
-- Last 10 films alphabetically (reverse order, take 10)
SELECT * FROM film
```

```
ORDER BY title DESC
LIMIT 10;
```

OFFSET

Skips a specified number of rows before starting to return results.

```
-- Skip the first 100 films, then return the next 10
SELECT * FROM film
LIMIT 10
OFFSET 100;
```

Can You Use OFFSET Without LIMIT?

No. `OFFSET` without `LIMIT` is invalid in MySQL. The database needs to know how many rows to return after skipping -- otherwise it would just return everything after the offset, making `OFFSET` pointless (you'd use `WHERE` instead).

However, you **can** use `LIMIT` without `OFFSET` (it just starts from the beginning).

Practical Use: Finding the Nth Highest Value

```
-- 2nd highest replacement_cost in the film table
SELECT * FROM film
ORDER BY replacement_cost DESC
LIMIT 1
OFFSET 1;
```

How it works:

```
Sorted DESC:      [29.99, 28.99, 27.99, 25.99, ...]
                  ^      ^
                  OFFSET 1 LIMIT 1
                  (skip)   (return this one)
```

```
-- 2nd highest payment amount (using DISTINCT to avoid ties)
SELECT DISTINCT amount FROM payment
ORDER BY amount DESC
LIMIT 1
OFFSET 1;
```

UPDATE -- Modifying Existing Data

Syntax

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
```

```
WHERE condition;
```

Example

```
UPDATE film
SET title = 'Ford vs Ferrari'
WHERE film_id = 1;

SELECT * FROM film WHERE film_id = 1;
```

SQL_SAFE_UPDATES

MySQL has a safety feature called `SQL_SAFE_UPDATES`. When enabled (default in Workbench):

- `UPDATE` and `DELETE` statements **must** include a `WHERE` clause that uses a **key column** (Primary Key or indexed column).
- This prevents accidental mass updates/deletes.

```
-- Check the current setting
SELECT @@sql_safe_updates;

-- Disable it (use with caution!)
SET SQL_SAFE_UPDATES = 0;

-- Re-enable it
SET SQL_SAFE_UPDATES = 1;
```

Why does it exist? Running `UPDATE film SET title = 'Oops'` without a `WHERE` clause would update **every** row in the table. `SQL_SAFE_UPDATES` prevents this mistake.

DELETE, TRUNCATE & DROP

These three commands all remove data, but they work very differently:

DELETE -- Remove Specific Rows

```
-- Delete a specific film
DELETE FROM film WHERE film_id = 1;

-- Delete all PG-13 films
DELETE FROM film WHERE rating = 'PG-13';

-- Delete ALL rows (but table structure remains)
DELETE FROM film;
```

- **DML** (Data Manipulation Language) operation.
- Can use `WHERE` to selectively delete rows.
- Can be **rolled back** inside a transaction.

- Slower for large deletes because it logs each row deletion.
- Auto-increment counters are **NOT** reset.

TRUNCATE -- Remove All Rows (Fast)

```
TRUNCATE TABLE film;
```

- **DDL** (Data Definition Language) operation.
- Removes **all** rows -- no `WHERE` clause allowed.
- **Cannot be rolled back** (in most configurations).
- Much **faster** than `DELETE FROM table` because it drops and recreates the table internally.
- Auto-increment counters **are reset** back to 1.

DROP -- Remove the Entire Table

```
DROP TABLE film;
```

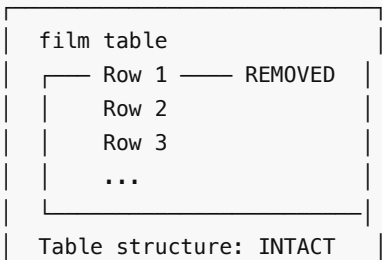
- **DDL** operation.
- Removes the **entire table** -- structure, data, indexes, constraints, everything.
- The table no longer exists after this.
- **Cannot be rolled back.**

Comparison Table

Feature	DELETE	TRUNCATE	DROP
What it removes	Specific rows (or all)	All rows	Entire table
WHERE clause	Yes	No	No
Table structure	Preserved	Preserved	Removed
Rollback	Yes (in transaction)	No	No
Speed	Slower (row-by-row)	Faster (bulk)	Fastest
Auto-increment	Not reset	Reset to 1	N/A (table gone)
Type	DML	DDL	DDL
Triggers fired	Yes	No	No

Visual

```
DELETE FROM film WHERE film_id = 1;
```



```
TRUNCATE TABLE film;
```

```
film table  
(all rows removed)
```

```
Table structure: INTACT  
Auto-increment: RESET
```

```
DROP TABLE film;
```

```
(table no longer exists)
```

SQL Query Execution Order

Understanding the order in which SQL processes a query helps explain why certain clauses must come in a specific order:

```
FROM      → Which table?  
WHERE     → Filter rows  
SELECT    → Choose columns  
DISTINCT  → Remove duplicates  
ORDER BY  → Sort results  
LIMIT     → Restrict output count  
OFFSET    → Skip rows
```

This is why:

- You can use column aliases in `ORDER BY` but not in `WHERE`.
- `DISTINCT` is applied before `ORDER BY`.
- `LIMIT` / `OFFSET` are always last.

Summary

Keyword/Clause	Purpose
DISTINCT	Remove duplicate rows from the result set.
ROUND()	Round a numeric value to N decimal places.
WHERE	Filter rows based on a condition.
AND/OR/NOT	Combine conditions. AND has higher precedence than OR.
IN	Shorthand for multiple OR conditions (equality only).
ORDER BY	Sort results (ASC default, DESC for reverse).

LIKE	Pattern matching with % (any chars) and _ (one char).
IS NULL	Check for NULL values (never use = NULL).
LIMIT	Restrict the number of rows returned.
OFFSET	Skip N rows before returning results (requires LIMIT).
UPDATE	Modify existing rows. Always use WHERE.
DELETE	Remove specific rows. Can be rolled back.
TRUNCATE	Remove all rows fast. Resets auto-increment. Cannot rollback.
DROP	Remove the entire table. Cannot rollback.
